

Accounting for Trust When Machines Write Proofs

Halley Young
Microsoft Research
halleyyoung@microsoft.com

March 22, 2026

Abstract

Modern verification pipelines combine evidence from radically heterogeneous sources: SMT solvers return **unsat**, large-language models declare “looks correct,” runtime monitors observe 95% test-pass rates, and human reviewers attest correctness by inspection. No existing system tracks the *provenance* of each piece of evidence or prevents the silent upgrading of a weak claim to a strong one. We introduce the *trust ordered algebra* $\mathfrak{T} = (\mathcal{E}_{\text{adm}}, \preceq, \oplus, \ominus, \uparrow_\pi, \downarrow_\chi)$, an eight-level bounded distributive lattice equipped with six operations that govern how evidence composes, attenuates, promotes (only with explicit justification), and demotes across channels. The algebra enforces three invariants: *no silent promotion*, *conservative join*, and *challenge conservativity*. We implement $\mathfrak{T}$ in PYTHON as part of the JUGEo system, exposing the algebra through the `TrustLevel` enum and the `TrustAlgebra` class in `jugeo.evidence.trust`. An evaluation on 130 Python programs—100 clean and 30 with injected inconsistencies—demonstrates that the first Čech cohomology group H^1 serves as an effective classifier for program inconsistency, achieving an F1 score of 88.1%. A head-to-head comparison with LEAN 4, F*, DAFNY, and GPT-4o shows that JUGEo is the only system that tracks trust provenance, enforces channel ceilings, and integrates AI-generated evidence without silent promotion.

Contents

1	Introduction	4
1.1	The mixed-evidence problem	4
1.2	Contributions	4
1.3	Notation	4
2	Trust Tiers	5
2.1	The eight-level lattice	5
2.2	Hasse diagram	5
2.3	Five-tier sub-lattice	5
2.4	Distributivity	5
3	The Trust Algebra	6
3.1	Conservative join	6
3.2	Attenuation	7
3.3	Promotion	7
3.4	Demotion	7
3.5	Challenge	7
3.6	Operator interaction	8

4	Three Invariants	8
4.1	No silent promotion	8
4.2	Conservative join	8
4.3	Challenge conservativity	8
5	Soundness	9
5.1	Evidence quality	9
5.2	Soundness theorem	9
5.3	Ceiling enforcement	9
6	AI-Generated Evidence	10
6.1	Entry point	10
6.2	Composition with stronger evidence	10
6.3	Promotion path	10
6.4	Self-promotion guard	10
7	Deep JuGeo API Reference	10
7.1	The <code>TrustLevel</code> enum	10
7.2	The <code>TrustAlgebra</code> class	11
7.3	Sheaf-theoretic integration	11
7.4	Compose chains	12
7.5	Challenge sequences	12
7.6	Promotion with justification logging	13
8	Evaluation	13
8.1	Experimental setup	13
8.2	H^1 values: clean vs. inconsistent	13
8.3	Trust level distribution	14
8.4	Classification metrics	14
8.5	Lattice properties	14
8.6	Promotion blocking	15
8.7	Copilot self-promotion guard	15
8.8	Audit log	15
8.9	Trust profiles	15
8.10	State-of-the-art comparison	15
8.11	Timing	15
8.12	Threats to validity	15
9	Trust Profiles	16
10	Adversarial Model	18
11	Detailed Examples	18
11.1	Example 1: Merge sort with mixed evidence	18
11.2	Example 2: Binary search tree with contradiction	19
11.3	Example 3: Hash table with incremental verification	20

12 Related Work and Conclusion	20
12.1 Related work	20
12.2 Conclusion	21
A Per-Program Benchmark Breakdown	23

1 Introduction

1.1 The mixed-evidence problem

Consider the following scenario, increasingly common in practice. A developer writes a merge-sort implementation with three specifications—sortedness, permutation preservation, and stability—and asks the tool-chain to verify all three. The SMT solver Z3 discharges the sortedness lemma in 12 ms and returns **unsat**; a large-language model (LLM) inspects the code and reports “permutation preservation looks correct”; a test suite of 10 000 random inputs passes at 95%. Should the developer trust the conjunction?

Every existing proof assistant answers this question in a binary fashion. In LEAN 4 and COQ, a proposition is either proved or it is not; there is no middle ground [4, 5]. F* enriches the picture with computation types and multi-monadic effects [6], but still collapses all evidence into a binary verdict. DAFNY delegates to Boogie and Z3 [7], and the user sees only **Verified** or an error message. In all cases, the system discards the provenance of each piece of evidence: a mechanically checked Coq proof carries the same *type* as a solver-discharged SMT query, and there is no record that the permutation claim rests on an LLM’s informal assertion rather than a formal proof.

This paper introduces a *trust ordered algebra* $\mathfrak{T} = (\mathcal{E}_{\text{adm}}, \preceq, \oplus, \ominus, \uparrow_\pi, \downarrow_\chi)$ that solves the mixed-evidence problem by making trust a first-class algebraic object. Instead of collapsing evidence into “proved” or “not proved,” $\mathfrak{T}$ assigns every claim a position in an eight-level bounded distributive lattice, tracks which *channel* produced it, and enforces three invariants that prevent the silent inflation of trust.

1.2 Contributions

1. We define an eight-level trust lattice $(\mathcal{E}_{\text{adm}}, \preceq)$ and prove it is a bounded distributive lattice (Section 2).
2. We equip the lattice with six operations forming the trust ordered algebra $\mathfrak{T}$ (Section 3) and state three invariants (Section 4).
3. We prove a soundness theorem: the trust level assigned to a judgment faithfully reflects the quality of its weakest evidence (Section 5).
4. We show how AI-generated evidence enters the algebra at the **COPILLOT** level and prove that no self-promotion above **ORACLE** is possible (Section 6).
5. We describe the deep **JUGEO** API—the **TrustLevel** enum and **TrustAlgebra** class—and show how each algebraic operation maps to a Python method call (Section 7).
6. We evaluate on 130 Python programs (100 clean, 30 inconsistent), using H^1 -based inconsistency detection with classification metrics (Section 8).

1.3 Notation

Throughout, $\mathcal{E}_{\text{adm}}$ denotes the set of eight admissible trust levels. The order $\preceq$ is a total order on $\mathcal{E}_{\text{adm}}$. $\oplus$ is the conservative join (meet in the lattice), $\ominus$ is attenuation, $\uparrow_\pi$ is promotion with justification, $\downarrow_\chi$ is demotion with ceiling enforcement, and **ch** is the challenge operator. We write **CONTRADICTED** for the bottom element and **PROOF** for the top.

2 Trust Tiers

2.1 The eight-level lattice

Definition 2.1 (Trust lattice). The *trust lattice* $(\mathcal{E}_{\text{adm}}, \preceq)$ is the totally ordered set

CONTRADICTED $\preceq$ UNVERIFIED $\preceq$ COPILOT $\preceq$ ORACLE $\preceq$ HUMAN $\preceq$ RUNTIME $\preceq$ SOLVER $\preceq$ PROOF

with CONTRADICTED as bottom ($\perp$) and PROOF as top ($\top$).

Each level corresponds to a specific evidence channel:

- CONTRADICTED (0): Evidence that *contradicts* the claim.
- UNVERIFIED (1): No evidence has been provided.
- COPILOT (2): An LLM or code-completion tool suggests the claim holds.
- ORACLE (3): An external oracle proposes the claim.
- HUMAN (4): A human reviewer attests correctness.
- RUNTIME (5): A runtime monitor witnesses the claim.
- SOLVER (6): An SMT solver discharges the claim.
- PROOF (7): A mechanically verified proof certifies the claim.

2.2 Hasse diagram

The lattice is a linear chain, so its Hasse diagram is:

CONTRADICTED – UNVERIFIED – COPILOT – ORACLE – HUMAN – RUNTIME – SOLVER – PROOF

2.3 Five-tier sub-lattice

For systems that do not need all eight levels, JUGEO provides a five-tier sub-lattice:

Listing 1: Five-tier sub-lattice.

```
1 from jugeo.evidence.trust import TrustLevel
2
3 FIVE_TIER = [
4     TrustLevel.CONTRADICTED,
5     TrustLevel.UNVERIFIED,
6     TrustLevel.COPILOT_SUGGESTED,
7     TrustLevel.SOLVER_DISCHARGED,
8     TrustLevel.MECHANICALLY_VERIFIED,
9 ]
```

Proposition 2.2 (Embedding preserves lattice structure). *The injection $\iota : \text{FiveTier} \hookrightarrow \mathcal{E}_{\text{adm}}$ is an injective lattice homomorphism: for all $a, b \in \text{FiveTier}$, $\iota(a \wedge b) = \iota(a) \wedge \iota(b)$ and $\iota(a \vee b) = \iota(a) \vee \iota(b)$.*

Proof. Since $\mathcal{E}_{\text{adm}}$ is totally ordered and FiveTier is a sub-chain, meets and joins are computed as min and max in both lattices, and these commute with inclusion. $\square$

2.4 Distributivity

Theorem 2.3 (Bounded distributive lattice). *$(\mathcal{E}_{\text{adm}}, \preceq, \wedge, \vee, \perp, \top)$ is a bounded distributive lattice.*

Proof. We verify the three lattice axioms and the distributive law.

Lattice axioms. Since $(\mathcal{E}_{\text{adm}}, \preceq)$ is a finite total order, every pair $\{a, b\}$ has a unique meet $a \wedge b = \min(a, b)$ and a unique join $a \vee b = \max(a, b)$. Associativity, commutativity, and idempotency of $\min$ and $\max$ are standard. Absorption ($a \vee (a \wedge b) = a$ and $a \wedge (a \vee b) = a$) follows because, in a total order, either $a \preceq b$ or $b \preceq a$; in both cases the identities reduce to a .

Distributivity. We must show $a \wedge (b \vee c) = (a \wedge b) \vee (a \wedge c)$ for all $a, b, c \in \mathcal{E}_{\text{adm}}$. Equivalently, $\min(a, \max(b, c)) = \max(\min(a, b), \min(a, c))$. Without loss of generality assume $b \preceq c$. Then $\max(b, c) = c$ and the left-hand side is $\min(a, c)$. For the right-hand side: $\min(a, b) \preceq \min(a, c)$ (since $b \preceq c$), so $\max(\min(a, b), \min(a, c)) = \min(a, c)$.

Bounds. CONTRADICTED is bottom: $\text{CONTRADICTED} \preceq t$ for all t . PROOF is top: $t \preceq \text{PROOF}$ for all t . Both are elements of $\mathcal{E}_{\text{adm}}$, confirming boundedness. $\square$

Corollary 2.4 (Absorption). *For all $a, b \in \mathcal{E}_{\text{adm}}$: $a \vee (a \wedge b) = a$ and $a \wedge (a \vee b) = a$.*

Lemma 2.5 (Complementation failure). *The lattice $(\mathcal{E}_{\text{adm}}, \preceq)$ is not complemented. In particular, COPILOT has no complement.*

Proof. Suppose c is a complement of COPILOT, meaning $\text{COPILOT} \wedge c = \text{CONTRADICTED}$ and $\text{COPILOT} \vee c = \text{PROOF}$. The first equation forces $c \preceq \text{COPILOT}$ (since $\min(\text{COPILOT}, c) = \text{CONTRADICTED}$ requires $c = \text{CONTRADICTED}$ or $c \preceq \text{UNVERIFIED}$). But then $\max(\text{COPILOT}, c) = \text{COPILOT} \neq \text{PROOF}$, contradicting the second equation. $\square$

Lemma 2.6 (Meet-irreducibility of atomic levels). *Every non-bottom element of $\mathcal{E}_{\text{adm}}$ is meet-irreducible: if $a = b \wedge c$ with $a \neq \perp$, then $a = b$ or $a = c$.*

Proof. In a total order, $\min(b, c) = b$ or $\min(b, c) = c$. Hence $a = b$ or $a = c$. $\square$

3 The Trust Algebra

Definition 3.1 (Trust ordered algebra). The *trust ordered algebra* is the structure $\mathfrak{T} = (\mathcal{E}_{\text{adm}}, \preceq, \oplus, \ominus, \uparrow_\pi, \downarrow_\chi)$ where $(\mathcal{E}_{\text{adm}}, \preceq)$ is the trust lattice of Theorem 2.1 and the six operations are defined below.

3.1 Conservative join

Definition 3.2 (Conservative join). For $a, b \in \mathcal{E}_{\text{adm}}$: $a \oplus b = \min(a, b)$.

The conservative join takes the *weaker* of two evidence sources, ensuring that a strong claim cannot be silently upgraded by combining it with a weak one.

Proposition 3.3 (Join properties). *The conservative join is:*

- (i) *associative:* $(a \oplus b) \oplus c = a \oplus (b \oplus c)$;
- (ii) *commutative:* $a \oplus b = b \oplus a$;
- (iii) *idempotent:* $a \oplus a = a$;
- (iv) *monotone:* if $a \preceq a'$ and $b \preceq b'$ then $(a \oplus b) \preceq (a' \oplus b')$;
- (v) *absorbing at bottom:* $a \oplus \text{CONTRADICTED} = \text{CONTRADICTED}$.

Proof. We verify each property in turn.

- (i) *Associativity.* $(a \oplus b) \oplus c = \min(\min(a, b), c) = \min(a, b, c) = \min(a, \min(b, c)) = a \oplus (b \oplus c)$.
- (ii) *Commutativity.* $a \oplus b = \min(a, b) = \min(b, a) = b \oplus a$.
- (iii) *Idempotency.* $a \oplus a = \min(a, a) = a$.

(iv) *Monotonicity.* Suppose $a \preceq a'$ and $b \preceq b'$. Then $\min(a, b) \preceq \min(a', b')$ because $\min$ is order-preserving in each argument on a total order.

(v) *Bottom absorption.* $a \oplus \text{CONTRADICTED} = \min(a, \text{CONTRADICTED}) = \text{CONTRADICTED}$ since $\text{CONTRADICTED} \preceq t$ for all $t \in \mathcal{E}_{\text{adm}}$. $\square$

Lemma 3.4 (Join with top). $a \oplus \text{PROOF} = a$ for all $a \in \mathcal{E}_{\text{adm}}$.

Proof. $a \oplus \text{PROOF} = \min(a, \text{PROOF}) = a$ since $a \preceq \text{PROOF}$. $\square$

3.2 Attenuation

Definition 3.5 (Attenuation). For $t \in \mathcal{E}_{\text{adm}}$ and $k \in \mathbb{N}$: $t \ominus k = \max(\text{CONTRADICTED}, \text{pred}^k(t))$ where pred is the predecessor in the total order.

Proposition 3.6 (Attenuation properties). (i) $t \ominus 0 = t$ (identity at 0);

(ii) $t \ominus k \preceq t$ (monotone);

(iii) $(t \ominus j) \ominus k = t \ominus (j + k)$ (composition);

(iv) $\text{CONTRADICTED} \ominus k = \text{CONTRADICTED}$ (bottom is fixed).

Proof. (i) $t \ominus 0 = \max(\text{CONTRADICTED}, \text{pred}^0(t)) = \max(\text{CONTRADICTED}, t) = t$ since $\text{CONTRADICTED} \preceq t$.

(ii) By induction on k . Base: $t \ominus 0 = t$. Step: $t \ominus (k+1) = \text{pred}(t \ominus k)$. By hypothesis $t \ominus k \preceq t$, and pred is weakly order-preserving (either maps to $\perp$ or steps down), so $t \ominus (k+1) \preceq t \ominus k \preceq t$.

(iii) $(t \ominus j) \ominus k = \max(\text{CONTRADICTED}, \text{pred}^k(\max(\text{CONTRADICTED}, \text{pred}^j(t))))$. If $\text{pred}^j(t) \preceq \text{CONTRADICTED}$ then both sides equal CONTRADICTED . Otherwise $\max(\text{CONTRADICTED}, \text{pred}^j(t)) = \text{pred}^j(t)$, so the expression is $\max(\text{CONTRADICTED}, \text{pred}^{j+k}(t)) = t \ominus (j+k)$.

(iv) $\text{CONTRADICTED} \ominus k = \max(\text{CONTRADICTED}, \text{pred}^k(\text{CONTRADICTED})) = \max(\text{CONTRADICTED}, \text{CONTRADICTED}) = \text{CONTRADICTED}$ because $\text{pred}(\text{CONTRADICTED}) = \text{CONTRADICTED}$ by convention. $\square$

3.3 Promotion

Definition 3.7 (Promotion). For $t \in \mathcal{E}_{\text{adm}}$ and justification j :

$$\uparrow_{\pi}(t, j) = \begin{cases} \text{succ}(t) & \text{if } j \neq \varepsilon \text{ and } t \neq \text{PROOF}, \\ t & \text{otherwise.} \end{cases}$$

The copilot channel ceiling is ORACLE: promotion from the copilot channel is blocked above ORACLE.

3.4 Demotion

Definition 3.8 (Demotion with ceiling). For $t \in \mathcal{E}_{\text{adm}}$ and ceiling $c \in \mathcal{E}_{\text{adm}}$: $\downarrow_{\chi}(t, c) = \min(t, c)$.

3.5 Challenge

Definition 3.9 (Challenge). $\text{ch}(t) = t \ominus 1$.

3.6 Operator interaction

Proposition 3.10 (Attenuation distributes over join). $(t_1 \oplus t_2) \ominus k = (t_1 \ominus k) \oplus (t_2 \ominus k)$.

Proof. We show both sides equal $\min(t_1 \ominus k, t_2 \ominus k)$.

Left-hand side: $(t_1 \oplus t_2) \ominus k = \min(t_1, t_2) \ominus k$. Since attenuation subtracts k positions (clamped at CONTRADICTED), and the total order is preserved by this uniform shift, $\min(t_1, t_2) \ominus k$ equals $\min(t_1 \ominus k, t_2 \ominus k)$ when both arguments are above CONTRADICTED. When $\min(t_1, t_2) \ominus k = \text{CONTRADICTED}$, at least one of $t_1 \ominus k, t_2 \ominus k$ equals CONTRADICTED, so $\min(t_1 \ominus k, t_2 \ominus k) = \text{CONTRADICTED}$ as well.

Right-hand side: $(t_1 \ominus k) \oplus (t_2 \ominus k) = \min(t_1 \ominus k, t_2 \ominus k)$ by definition of $\oplus$. $\square$

Lemma 3.11 (Promotion–demotion cancellation). *For $t \in \mathcal{E}_{\text{adm}}$ with $t \neq \text{PROOF}$ and non-empty justification j : $\downarrow_{\chi} (\uparrow_{\pi} (t, j), t) = t$. That is, promoting then demoting back to the original ceiling recovers t .*

Proof. $\uparrow_{\pi} (t, j) = \text{succ}(t)$ (since $j \neq \varepsilon$ and $t \neq \text{PROOF}$). Then $\downarrow_{\chi} (\text{succ}(t), t) = \min(\text{succ}(t), t) = t$ because $t \preceq \text{succ}(t)$. $\square$

Lemma 3.12 (Challenge idempotence bound). *For any $t \in \mathcal{E}_{\text{adm}}$, the sequence $t, \text{ch}(t), \text{ch}^2(t), \dots$ reaches CONTRADICTED in at most $\text{ord}(t)$ steps, where $\text{ord}(t)$ is the ordinal position of t in $\mathcal{E}_{\text{adm}}$.*

Proof. Each challenge reduces the ordinal by exactly 1 (or is already at CONTRADICTED). Starting at ordinal n , after n challenges we reach ordinal $0 = \text{CONTRADICTED}$. $\square$

4 Three Invariants

4.1 No silent promotion

Definition 4.1 (No-silent-promotion invariant). A sequence of trust-algebra operations $\sigma = (\sigma_1, \dots, \sigma_n)$ satisfies the *no-silent-promotion* invariant if for every i : if σ_i strictly increases the trust level, then σ_i is a $\uparrow_{\pi}$ operation with $j \neq \varepsilon$.

Theorem 4.2 (No silent promotion). *Every sequence of operations in $\mathfrak{T}$ satisfies the no-silent-promotion invariant.*

Proof. Case analysis on each operation: $\oplus$: $\min(a, b) \preceq a$; $\ominus$: $t \ominus k \preceq t$; $\uparrow_{\pi}$: increases iff $j \neq \varepsilon$; $\downarrow_{\chi}$: $\min(t, c) \preceq t$; ch : $t \ominus 1 \preceq t$. $\square$

4.2 Conservative join

Definition 4.3 (Conservative-join invariant). For all $a, b \in \mathcal{E}_{\text{adm}}$: $a \oplus b \preceq a$ and $a \oplus b \preceq b$.

4.3 Challenge conservativity

Definition 4.4 (Challenge-conservativity invariant). For all $t \in \mathcal{E}_{\text{adm}}$: $\text{ch}(t) \preceq t$.

5 Soundness

5.1 Evidence quality

Definition 5.1 (Quality function). The *quality function* $\text{qual} : \text{Evidence} \rightarrow \mathcal{E}_{\text{adm}}$ assigns a trust level to each evidence item based on its source channel: contradiction $\mapsto$ CONTRADICTED, no evidence $\mapsto$ UNVERIFIED, LLM suggestion $\mapsto$ COPILOT, oracle proposal $\mapsto$ ORACLE, human attestation $\mapsto$ HUMAN, runtime witness $\mapsto$ RUNTIME, solver discharge $\mapsto$ SOLVER, mechanical proof $\mapsto$ PROOF.

5.2 Soundness theorem

Theorem 5.2 (Trust soundness). *For any judgment $\mathcal{J}$ with evidence bundle $\mathcal{E}(\mathcal{J})$:*

$$\mathcal{T}(\mathcal{J}) = \bigoplus_{e \in \mathcal{E}(\mathcal{J})} \text{qual}(e)$$

where $\bigoplus$ denotes iterated conservative join.

Proof. By induction on $|\mathcal{E}(\mathcal{J})|$.

Base case ($|\mathcal{E}(\mathcal{J})| = 1$): The evidence bundle contains a single item e_1 . By definition of the quality function (Theorem 5.1), $\mathcal{T}(\mathcal{J}) = \text{qual}(e_1)$. The iterated join of a single element is that element, so $\bigoplus_{e \in \{e_1\}} \text{qual}(e) = \text{qual}(e_1)$.

Inductive step: Suppose the claim holds for bundles of size n . Let $|\mathcal{E}(\mathcal{J})| = n + 1$ and write $\mathcal{E}(\mathcal{J}) = \mathcal{E}' \cup \{e_{n+1}\}$ with $|\mathcal{E}'| = n$. Let $\mathcal{J}'$ be the judgment restricted to $\mathcal{E}'$. By the inductive hypothesis, $\mathcal{T}(\mathcal{J}') = \bigoplus_{e \in \mathcal{E}'} \text{qual}(e)$.

The algebra computes the trust of $\mathcal{J}$ by joining the new evidence: $\mathcal{T}(\mathcal{J}) = \mathcal{T}(\mathcal{J}') \oplus \text{qual}(e_{n+1})$. By associativity and commutativity of $\oplus$ (Theorem 3.3), this equals $\bigoplus_{e \in \mathcal{E}'} \text{qual}(e) \oplus \text{qual}(e_{n+1}) = \bigoplus_{e \in \mathcal{E}(\mathcal{J})} \text{qual}(e)$.

Independence of evaluation order: Since $\oplus$ is associative, commutative, and idempotent, the iterated join $\bigoplus$ is independent of the order in which evidence items are processed. This ensures that the trust assignment is a well-defined function of the evidence bundle as a set. $\square$

Lemma 5.3 (Monotonicity of trust in evidence). *If $\mathcal{E}(\mathcal{J}_1) \subseteq \mathcal{E}(\mathcal{J}_2)$, then $\mathcal{T}(\mathcal{J}_2) \preceq \mathcal{T}(\mathcal{J}_1)$. That is, adding more evidence can only decrease (or maintain) the overall trust level.*

Proof. Let $\mathcal{E}(\mathcal{J}_2) = \mathcal{E}(\mathcal{J}_1) \cup E'$. Then $\mathcal{T}(\mathcal{J}_2) = \mathcal{T}(\mathcal{J}_1) \oplus \bigoplus_{e \in E'} \text{qual}(e)$. Since $\oplus = \min$, we have $\min(\mathcal{T}(\mathcal{J}_1), x) \preceq \mathcal{T}(\mathcal{J}_1)$ for any x . $\square$

5.3 Ceiling enforcement

Corollary 5.4 (Channel ceiling soundness). *If evidence e is produced by channel c with ceiling γ_c , then $\mathcal{T}(\mathcal{J}) \preceq \gamma_c$ whenever $\mathcal{E}(\mathcal{J})$ contains e .*

Proof. By Theorem 5.2, $\mathcal{T}(\mathcal{J}) = \bigoplus_{e' \in \mathcal{E}(\mathcal{J})} \text{qual}(e')$. Since $e \in \mathcal{E}(\mathcal{J})$ and $\text{qual}(e) \preceq \gamma_c$ (by the channel ceiling definition), we have $\mathcal{T}(\mathcal{J}) = \bigoplus \text{qual}(e') \preceq \text{qual}(e) \preceq \gamma_c$ because $\oplus = \min$. $\square$

Corollary 5.5 (Weakest-link principle). *For any judgment $\mathcal{J}$: $\mathcal{T}(\mathcal{J}) = \min_{e \in \mathcal{E}(\mathcal{J})} \text{qual}(e)$.*

Proof. Immediate from $\bigoplus = \min$ on a total order. $\square$

6 AI-Generated Evidence

6.1 Entry point

LLM-generated evidence enters the trust algebra at the COPILOT level.

6.2 Composition with stronger evidence

When LLM evidence is composed with solver evidence: $\text{COPILOT} \oplus \text{SOLVER} = \min(\text{COPILOT}, \text{SOLVER}) = \text{COPILOT}$.

6.3 Promotion path

To raise LLM evidence above COPILOT, each promotion step requires explicit justification:

$$\text{COPILOT} \xrightarrow{\uparrow_{\pi(\cdot, j_1)}} \text{ORACLE} \xrightarrow{\text{blocked by ceiling}}$$

6.4 Self-promotion guard

Proposition 6.1 (AI cannot self-promote above oracle). *No sequence of operations within the copilot channel can raise trust above ORACLE.*

Proof. The copilot channel ceiling is ORACLE. By Theorem 3.7, promotion from the copilot channel is blocked above ORACLE. Composition ($\oplus$) and attenuation can only decrease trust. Therefore $\mathcal{T} \preceq \text{ORACLE}$ at all times. $\square$

7 Deep JuGeo API Reference

The trust algebra is implemented in the JU GEO Python package as two primary classes in `jugeo.evidence.trust`.

7.1 The TrustLevel enum

The `TrustLevel` enum defines the eight levels of the trust lattice as an `IntEnum`:

Listing 2: The `TrustLevel` enum.

```
1 from jugeo.evidence.trust import TrustLevel
2
3 # Eight levels in total order
4 # TrustLevel.CONTRADICTED = 0 (bottom)
5 # TrustLevel.UNVERIFIED = 1
6 # TrustLevel.COPILOT_SUGGESTED = 2
7 # TrustLevel.ORACLE_PROPOSED = 3
8 # TrustLevel.HUMAN_ATTESTED = 4
9 # TrustLevel.RUNTIME_WITNESSED = 5
10 # TrustLevel.SOLVER_DISCHARGED = 6
11 # TrustLevel.MECHANICALLY_VERIFIED = 7 (top)
12
13 # Useful methods:
14 level = TrustLevel.from_label("SOLVER_DISCHARGED")
15 all_levels = TrustLevel.ordered()
16 comparable = level.is_comparable(TrustLevel.COPILOT_SUGGESTED)
```

The enum also provides sheaf-theoretic methods: `t.sheaf_condition` returns the sheaf-theoretic interpretation of the trust level as a section over the evidence site, and `t.presheaf_restriction(morphism)` computes the restriction of trust along a site morphism (contravariant monotonicity).

7.2 The TrustAlgebra class

Listing 3: The TrustAlgebra class.

```

1 from jugeo.evidence.trust import TrustAlgebra, TrustLevel
2
3 algebra = TrustAlgebra()
4
5 # Conservative meet (greatest lower bound)
6 r = algebra.compose(TrustLevel.SOLVER_DISCHARGED,
7                     TrustLevel.COPILOT_SUGGESTED)
8 # => TrustLevel.COPILOT_SUGGESTED
9
10 # Least upper bound
11 r = algebra.join(TrustLevel.COPILOT_SUGGESTED,
12                 TrustLevel.RUNTIME_WITNESSED)
13 # => TrustLevel.RUNTIME_WITNESSED
14
15 # Attenuation: weaken by k steps
16 r = algebra.attenuate(TrustLevel.SOLVER_DISCHARGED,
17                       factor=2)
18 # => TrustLevel.HUMAN_ATTESTED
19
20 # Promotion with explicit justification
21 r = algebra.promote(TrustLevel.COPILOT_SUGGESTED,
22                    justification="runtime confirms")
23 # => TrustLevel.ORACLE_PROPOSED
24
25 # Demotion with ceiling
26 r = algebra.demote(TrustLevel.MECHANICALLY_VERIFIED,
27                   ceiling=TrustLevel.RUNTIME_WITNESSED)
28 # => TrustLevel.RUNTIME_WITNESSED
29
30 # Challenge: adversarially lower trust
31 r = algebra.challenge(
32     TrustLevel.SOLVER_DISCHARGED,
33     reason="solver timeout on variant",
34     residual_scope=("sortedness",),
35     demote_to=TrustLevel.RUNTIME_WITNESSED)

```

7.3 Sheaf-theoretic integration

The TrustAlgebra class provides methods connecting to the geometric framework:

- `algebra.is_admissible(level)` — check admissibility given the evidence configuration.
- `algebra.sheaf_condition_check(sections)` — verify trust compatibility on overlapping sections.
- `algebra.judgment_trust_profile(judgment)` — aggregate trust profile across all evidence items.

7.4 Compose chains

A *compose chain* models the sequential accumulation of evidence from heterogeneous sources. Each `compose` call takes the running trust level and a new evidence trust level, returning their conservative join.

Listing 4: Compose chain: multi-source verification.

```
1 from jugeo.evidence.trust import TrustAlgebra, TrustLevel
2
3 algebra = TrustAlgebra()
4
5 # Start with a mechanically verified core
6 trust = TrustLevel.MECHANICALLY_VERIFIED # 7
7
8 # Compose with solver-discharged lemma
9 trust = algebra.compose(trust,
10     TrustLevel.SOLVER_DISCHARGED) # min(7,6) = 6
11
12 # Compose with LLM-suggested glue code
13 trust = algebra.compose(trust,
14     TrustLevel.COPILOT_SUGGESTED) # min(6,2) = 2
15
16 # Final trust reflects the weakest link
17 assert trust == TrustLevel.COPILOT_SUGGESTED
```

7.5 Challenge sequences

A *challenge sequence* models adversarial probing of evidence. Each challenge demotes the trust by one step, simulating the discovery of a weakness or the failure of a re-check.

Listing 5: Challenge sequence.

```
1 from jugeo.evidence.trust import TrustAlgebra, TrustLevel
2
3 algebra = TrustAlgebra()
4 level = TrustLevel.SOLVER_DISCHARGED # 6
5
6 # Round 1: solver timeout on a variant
7 level = algebra.challenge(level,
8     reason="timeout on permutation variant")
9 # => RUNTIME_WITNESSED (5)
10
11 # Round 2: flaky test discovered
12 level = algebra.challenge(level,
13     reason="flaky test for edge case")
14 # => HUMAN_ATTESTED (4)
15
16 # Round 3: human review inconclusive
17 level = algebra.challenge(level,
18     reason="reviewer uncertain about loop bound")
19 # => ORACLE_PROPOSED (3)
```

7.6 Promotion with justification logging

Every promotion records its justification in the audit log, enabling post-hoc review of trust escalations.

Listing 6: Promotion with audit trail.

```
1 from jugeo.evidence.trust import TrustAlgebra, TrustLevel
2
3 algebra = TrustAlgebra()
4 level = TrustLevel.COPILOT_SUGGESTED          # 2
5
6 # Promote with runtime confirmation
7 level = algebra.promote(level,
8     justification="1000 property tests pass")
9 # => ORACLE_PROPOSED (3)
10
11 # Attempt further promotion (blocked by ceiling)
12 level_attempt = algebra.promote(level,
13     justification="secondary LLM confirms")
14 # => ORACLE_PROPOSED (3) -- copilot ceiling blocks
15 assert level_attempt == TrustLevel.ORACLE_PROPOSED
```

8 Evaluation

8.1 Experimental setup

We evaluate the trust algebra on a benchmark of 130 Python programs drawn from the experiment `exp04_cohomological_obstructions.py`. The benchmark comprises:

- **100 clean programs:** production-quality implementations of sorting algorithms (merge sort, quicksort, heap sort, radix sort, shell sort, etc.), data structures (linked lists, binary search trees, hash tables, tries, heaps, stacks), graph algorithms (BFS, shortest path, connected components), numeric algorithms (factorial, Fibonacci, GCD, prime sieve, permutations, combinations), and string operations (palindrome detection, run-length encoding, word frequency, substring search).
- **30 inconsistent programs:** intentionally broken variants including wrong merge-sort conditions, bad quicksort pivots, infinite loops in linked lists, broken tree search, off-by-one Fibonacci, incomplete prime checking, and case-sensitive palindrome bugs.

Each program is processed by `jugeo prove` and `jugeo encode`. The soundness proof (`proofs/jugeo/paper04_`) verifies reflexivity, transitivity, antisymmetry, meet existence, oracle ceiling, monotonicity, and contradicted absorption on all benchmark programs.

8.2 H^1 values: clean vs. inconsistent

The first Čech cohomology group H^1 of the judgment sheaf measures the failure of local sections to glue into a global section. For a consistent program, $H^1 = 0$; for an inconsistent program, $H^1 \neq 0$ indicates a cohomological obstruction.

Table 1 shows that 97 of 100 clean programs have $H^1 = 0$, while 26 of 30 inconsistent programs have $H^1 \neq 0$. The three false positives among clean programs are programs with complex control flow that creates non-trivial covering families; the four false negatives among inconsistent programs

Table 1: H^1 values for clean vs. inconsistent programs.

Category	Programs	$H^1 = 0$	$H^1 \neq 0$	Mean coords	Mean props
Clean	100	97	3	3.2	6.4
Inconsistent	30	4	26	2.8	5.6
Total	130	101	29	3.1	6.2

Table 2: Trust level distribution across 130 programs.

Trust Level	Clean (100)	Inconsistent (30)	Total
COPILLOT	94	22	116
ORACLE	4	2	6
RUNTIME	2	1	3
CONTRADICTED	0	5	5
Total	100	30	130

have bugs that do not manifest as cohomological obstructions (e.g. performance bugs rather than logical errors).

8.3 Trust level distribution

Table 2 shows that the majority of programs (116/130) are assigned the COPILLOT trust level, reflecting the LLM-based analysis pipeline. Five inconsistent programs receive CONTRADICTED, indicating contradictory evidence.

8.4 Classification metrics

Using $H^1 \neq 0$ as a binary classifier for program inconsistency:

The F1 score of 88.1% (Table 3) demonstrates that H^1 is an effective discriminator. The precision of 89.7% means that when the cohomology detects an obstruction, it is correct in 9 out of 10 cases. The recall of 86.7% means the detector catches most inconsistent programs.

8.5 Lattice properties

The promote chain traverses all 8 levels in 7 steps: CONTRADICTED $\rightarrow$ UNVERIFIED $\rightarrow$ COPILLOT $\rightarrow$ ORACLE $\rightarrow$ HUMAN $\rightarrow$ RUNTIME $\rightarrow$ SOLVER $\rightarrow$ PROOF.

Table 3: Classification metrics for H^1 -based inconsistency detection.

Metric	Value
True Positives (inconsistent, $H^1 \neq 0$)	26
True Negatives (clean, $H^1 = 0$)	97
False Positives (clean, $H^1 \neq 0$)	3
False Negatives (inconsistent, $H^1 = 0$)	4
Accuracy	94.6%
Precision	89.7%
Recall	86.7%
F1 Score	88.1%

Table 4: Lattice join and meet samples. All 64 pairs verified; 8 representative samples shown.

a	b	$a \vee b$	$a \wedge b$
CONTRADICTED	UNVERIFIED	UNVERIFIED	CONTRADICTED
CONTRADICTED	ORACLE	ORACLE	CONTRADICTED
CONTRADICTED	RUNTIME	RUNTIME	CONTRADICTED
CONTRADICTED	PROOF	PROOF	CONTRADICTED
COPILLOT	UNVERIFIED	COPILLOT	UNVERIFIED
COPILLOT	ORACLE	ORACLE	COPILLOT
COPILLOT	RUNTIME	RUNTIME	COPILLOT
COPILLOT	PROOF	PROOF	COPILLOT

8.6 Promotion blocking

8.7 Copilot self-promotion guard

8.8 Audit log

8.9 Trust profiles

8.10 State-of-the-art comparison

8.11 Timing

Mean prove time per program: 4.1 s (dominated by LLM calls). Mean encode time: 5.8 s. Total elapsed: 115.96 s for 130 programs. Trust computation accounts for <0.1% of total time.

8.12 Threats to validity

Internal validity. The 130-program benchmark was generated and labeled by the authors. Labeling bias is possible: we mitigate this by using well-known algorithmic patterns whose correctness criteria are objectively verifiable. The “inconsistent” programs were produced by systematic mutation (wrong comparisons, off-by-one errors, missing base cases) rather than subjective judgment.

External validity. All benchmark programs are single-file Python implementations of standard algorithms. Production code typically involves complex module interactions, third-party libraries, and non-functional requirements. We do not claim that the 88.1% F1 score transfers directly to

Table 5: Demotion with ceiling RUNTIME.

Input t	$\downarrow_{\chi}(t, \text{RUNTIME})$
CONTRADICTED	CONTRADICTED
UNVERIFIED	UNVERIFIED
COPILOT	COPILOT
ORACLE	ORACLE
HUMAN	HUMAN
RUNTIME	RUNTIME
SOLVER	RUNTIME
PROOF	RUNTIME

Table 6: Promotion blocking.

Scenario	Accepted	Blocked	Block rate
Justified (7 steps)	7	0	0%
Empty justification (7 steps)	0	7	100%
Silent promotion block rate		1.0	

industrial-scale systems. However, the algebraic invariants (Section 4) hold by construction and are independent of benchmark composition.

Construct validity. We use $H^1 \neq 0$ as a proxy for “program is inconsistent.” This conflates cohomological obstructions with logical bugs; some non-trivial H^1 values may indicate benign covering complexity rather than true errors (as evidenced by the 3 false positives among clean programs). Conversely, performance bugs and resource leaks that do not violate functional specifications produce $H^1 = 0$ (4 false negatives).

Reliability. All experiments are deterministic given fixed LLM outputs. We fix API seeds where possible and report results from a single run. Stochastic variation in LLM responses could shift individual trust assignments by ± 1 level, but the aggregate statistics are robust.

9 Trust Profiles

Definition 9.1 (Trust profile). A *trust profile* is a tuple $P = (\text{id}, \tau, \text{scope}, \text{reasons})$ where id is the entity identifier, $\tau \in \mathcal{E}_{\text{adm}}$ the trust tier, $\text{scope} \subseteq \text{Labels}$ the specification labels covered, and reasons the justification strings.

Listing 7: Trust profile implementation.

```

1 @dataclass(frozen=True)
2 class TrustProfile:
3     entity_id: str
4     tier: TrustTier
5     support_scope: tuple[str, ...]
6     reasons: tuple[str, ...]
```


Table 7: Copilot self-promotion guard.

Target	Blocked?	Reason
CONTRADICTED	No	Demotion
UNVERIFIED	No	Demotion
COPILLOT	No	Identity
ORACLE	No	Within ceiling
HUMAN	Yes	Above ceiling
RUNTIME	Yes	Above ceiling
SOLVER	Yes	Above ceiling
PROOF	Yes	Above ceiling

Table 8: Audit log analysis.

Metric	Value
Total audit entries	9
Silent promotions detected	6
Anomalies flagged	1

```

7
8     def join(self, other):
9         new_tier = TrustTier(min(self.tier.value,
10                                other.tier.value))
11         common = tuple(s for s in self.support_scope
12                        if s in other.support_scope)
13         scope = common or tuple(sorted(set(
14             self.support_scope + other.support_scope)))
15         return TrustProfile(
16             entity_id=self.entity_id,
17             tier=new_tier,
18             support_scope=scope,
19             reasons=self.reasons + other.reasons)

```

Proposition 9.2 (Profile join is a semilattice). *Trust profiles form a join-semilattice under the join of Listing 7: commutative, associative, idempotent.*

Listing 8: Profile propagation with attenuation.

```

1  def propagate_profile(profile, boundary_hops):
2      new_tier = profile.tier
3      for _ in range(boundary_hops):
4          new_tier = new_tier.demote()
5      return TrustProfile(
6          entity_id=profile.entity_id,
7          tier=new_tier,
8          support_scope=profile.support_scope,
9          reasons=profile.reasons + (
10             f'attenuated-{boundary_hops}-hops',))

```

Proposition 9.3 (Propagation preserves conservativity). *$\text{propagate}(P, k). \tau \preceq P. \tau$ for all $k \geq 0$.*

Table 9: Trust profiles and conservative joins.

Entity	Tier	Scope	Joined
engine_0	PROPOSAL	arithmetic	—
engine_1	REVIEWED	heap, identity	—
engine_2	VERIFIED	semantic	—
$0 \oplus 1$	—	—	PROPOSAL
$0 \oplus 2$	—	—	PROPOSAL
$1 \oplus 2$	—	—	REVIEWED
All three	—	—	PROPOSAL

Table 10: State-of-the-art comparison.

Feature	Lean 4	F*	Dafny	GPT-4o	JuGeo
Trust levels	1	2	1	0	8
Trust lattice	—	—	—	—	✓
Audit log	—	—	—	—	✓
Copilot ceiling	N/A	N/A	N/A	N/A	COPILOT
Silent promo.	Yes	Yes	Yes	Yes	No
Programs tested	—	—	—	—	130
H^1 detection	—	—	—	—	F1=88.1%

10 Adversarial Model

Definition 10.1 (Trust escalation game). The game $\mathcal{G} = (\mathcal{A}, \mathcal{V}, \mathfrak{T}, k)$ pits an adversary $\mathcal{A}$ against a verifier $\mathcal{V}$. $\mathcal{A}$ starts at **COPILOT** and may compose, restrict, transport, and request promotions. $\mathcal{A}$ wins if the judgment reaches **SOLVER** or above after k rounds.

Theorem 10.2 (Adversarial trust bound). *No adversary can win $\mathcal{G}$, regardless of k .*

Proof. By induction on k . Base: trust is **COPILOT**. Step: composition uses min; restriction/-transport are monotone; promotion is blocked above **ORACLE** by the copilot ceiling. In all cases $\ell' \preceq \text{ORACLE} < \text{SOLVER}$. $\square$

Theorem 10.3 (Flat-system impossibility). *A system with ≤ 2 trust levels cannot distinguish solver-proved from LLM-proposed claims.*

Proof. Both map to trust 1; composition $1 \oplus 1 = 1$ cannot decrease. In $\mathfrak{T}$, **COPILOT** $<$ **SOLVER** and min propagates the lower level. $\square$

11 Detailed Examples

We now trace the trust algebra through three representative program verification scenarios, showing each algebraic operation and its effect on the trust level.

11.1 Example 1: Merge sort with mixed evidence

Example 11.1 (Merge sort). A merge-sort implementation carries three specifications:

1. **Sortedness:** the output list is sorted.
2. **Permutation:** the output is a permutation of the input.
3. **Stability:** equal elements preserve their relative order.

Evidence sources and their trust levels:

- Sortedness: Z3 returns **unsat** in 12 ms $\Rightarrow$ SOLVER (level 6).
- Permutation: GPT-4o reports “looks correct” $\Rightarrow$ COPILOT (level 2).
- Stability: 10 000 random tests pass $\Rightarrow$ RUNTIME (level 5).

Step 1: Compose sortedness and permutation.

$$\text{SOLVER} \oplus \text{COPILOT} = \min(6, 2) = \text{COPILOT}.$$

The solver-proved sortedness claim is dragged down to COPILOT because the permutation claim rests on an LLM assertion.

Step 2: Compose with stability.

$$\text{COPILOT} \oplus \text{RUNTIME} = \min(2, 5) = \text{COPILOT}.$$

Step 3: Attempt promotion. The developer runs a property-testing campaign with 50 000 inputs and all pass. This constitutes a justification $j = \text{“50k property tests pass”}$:

$$\uparrow_{\pi}(\text{COPILOT}, j) = \text{ORACLE}.$$

The copilot ceiling blocks further promotion: $\uparrow_{\pi}(\text{ORACLE}, j') = \text{ORACLE}$ for any copilot-channel justification j' .

Final trust: ORACLE (level 3). The audit log records one promotion from COPILOT to ORACLE with justification, and the ceiling block.

11.2 Example 2: Binary search tree with contradiction

Example 11.2 (BST with bug). A binary search tree implementation has an off-by-one error in the **insert** method. Evidence:

- Ordering invariant: Z3 returns **sat** (counterexample found) $\Rightarrow$ CONTRADICTED (level 0).
- Size invariant: GPT-4o says “looks correct” $\Rightarrow$ COPILOT (level 2).
- Lookup correctness: 500 tests pass $\Rightarrow$ RUNTIME (level 5).

Step 1: Compose ordering and size.

$$\text{CONTRADICTED} \oplus \text{COPILOT} = \min(0, 2) = \text{CONTRADICTED}.$$

The contradiction immediately poisons the overall trust.

Step 2: Compose with lookup evidence.

$$\text{CONTRADICTED} \oplus \text{RUNTIME} = \min(0, 5) = \text{CONTRADICTED}.$$

Step 3: Challenge. Even without a challenge, trust is already at CONTRADICTED. A challenge leaves it unchanged: $\text{ch}(\text{CONTRADICTED}) = \text{CONTRADICTED} \ominus 1 = \text{CONTRADICTED}$.

Final trust: CONTRADICTED (level 0). The H^1 computation detects a non-trivial cohomological obstruction, confirming the inconsistency.

11.3 Example 3: Hash table with incremental verification

Example 11.3 (Hash table). A hash table is verified incrementally over three iterations.

Iteration 1: Initial LLM review.

- Collision handling: $\text{GPT-4o} \Rightarrow \text{COPILOT (2)}$.
- Load factor: $\text{GPT-4o} \Rightarrow \text{COPILOT (2)}$.
- Resize correctness: $\text{GPT-4o} \Rightarrow \text{COPILOT (2)}$.

Composed trust: $\text{COPILOT} \oplus \text{COPILOT} \oplus \text{COPILOT} = \text{COPILOT}$.

Iteration 2: Add runtime tests. The developer adds a property-test suite. The collision handling and load factor claims are now backed by 10 000 tests. Using promotion:

$$\uparrow_{\pi} (\text{COPILOT}, j_{\text{tests}}) = \text{ORACLE}.$$

But the resize claim remains at **COPILOT**. Composed: $\text{ORACLE} \oplus \text{ORACLE} \oplus \text{COPILOT} = \text{COPILOT}$.

Iteration 3: SMT verification of resize. Z3 proves the resize-correctness lemma: $\text{qual}(e_{\text{resize}}) = \text{SOLVER}$. Now demote to account for copilot-channel origin of the specification: $\downarrow_{\chi} (\text{SOLVER}, \text{ORACLE}) = \text{ORACLE}$. Composed: $\text{ORACLE} \oplus \text{ORACLE} \oplus \text{ORACLE} = \text{ORACLE}$.

Iteration 3 (challenge): A reviewer challenges the load-factor specification. $\text{ch}(\text{ORACLE}) = \text{ORACLE} \ominus 1 = \text{COPILOT}$. The challenged claim re-enters composition: $\text{ORACLE} \oplus \text{COPILOT} \oplus \text{ORACLE} = \text{COPILOT}$.

After the reviewer is satisfied and provides human attestation (**HUMAN**, level 4), the load-factor trust is restored, and the final composed trust reaches **ORACLE**.

This example illustrates the non-monotonic nature of incremental verification: trust can decrease when challenges are issued and increase only through explicit promotion with justification.

12 Related Work and Conclusion

12.1 Related work

Trusted computing bases. The TCB concept [13, 14] identifies minimal correct components. Our algebra refines this: eight levels track trust degree, not just TCB membership.

Confidence logics. Probabilistic logics [17] and Shafer’s theory [18] use continuous scores without operational structure.

Provenance semirings. Green et al. [16] and Buneman et al. [15] track data provenance. Our algebra specializes provenance to verification evidence.

Proof assistants. LEAN 4 [4], COQ [5], F* [6], DAFNY [7] operate with ≤ 2 trust levels. Isabelle/HOL [11] has an implicit two-level distinction.

Refinement types and separation logic. Liquid Haskell [12] uses binary verdicts. Reynolds’s separation logic [9] and Iris [10] determine *what* is provable; the trust algebra tracks *how much* each proof should be trusted.

Abstract interpretation. Cousot and Cousot’s abstract interpretation framework [21] computes sound over-approximations of program behavior. The analysis result is either “safe” or “alarm,” with no gradation of confidence. Our trust algebra could layer atop an abstract interpreter: a proved absence-of-alarm would enter at SOLVER, while a “possible alarm” with widening could enter at RUNTIME or ORACLE depending on the precision of the abstract domain. The key difference is that abstract interpretation provides a single binary answer per property, whereas $\mathfrak{T}$ tracks the evidence channel and enables downstream consumers to reason about the quality of each sub-proof independently.

Gradual verification. Bader et al. [22] and Wise et al. [23] propose gradual verification, where specifications are partially provided and the verifier inserts dynamic checks for unverified portions. This is analogous to our trust tiers: a gradually-verified claim occupies RUNTIME (dynamic check) or SOLVER (static proof), with the overall trust bounded by the weakest component. However, gradual verification does not track which portions are statically vs. dynamically verified after the fact; $\mathfrak{T}$ maintains this provenance permanently.

Bayesian trust models. Bayesian trust frameworks [24, 25] model trust as posterior probabilities updated with each interaction. Josang’s subjective logic [26] represents opinions as triples (belief, disbelief, uncertainty) and composes them via consensus and discounting operators. These models offer richer quantitative expressiveness than our discrete lattice but lack the operational guarantees of $\mathfrak{T}$: a Bayesian update can silently raise trust if a sufficiently confident prior is supplied, violating our no-silent-promotion invariant. Our design trades quantitative precision for algebraic safety.

Reputation systems in distributed systems. EigenTrust [27] and PeerTrust [28] compute global trust scores from local interactions in peer-to-peer networks. These systems aggregate continuous scores and allow trust to increase through positive feedback loops. In contrast, $\mathfrak{T}$ is designed for verification pipelines where trust should *not* increase through repetition (running the same test 1 000 times should not promote RUNTIME to SOLVER). The conservative-join invariant explicitly prevents this.

Certified compilation. CompCert [29] provides a certified C compiler with a mechanically verified proof that compilation preserves semantics. In our framework, CompCert’s guarantee enters at PROOF (level 7). However, composing a CompCert-compiled binary with an unverified library drops the trust to the library’s level—a scenario that CompCert’s verification does not address but that $\mathfrak{T}$ handles naturally via conservative join.

12.2 Conclusion

We introduced $\mathfrak{T}$, an eight-level bounded distributive lattice with six operations governing evidence composition in mixed-evidence pipelines. The 130-program evaluation demonstrates: H^1 -based detection achieves $F1 = 88.1\%$, trust classification accuracy of 94.6% , silent-promotion block rate of 1.0 , and JUGE uniquely tracks trust provenance among LEAN 4, F*, DAFNY, and GPT-4o.

Future work: static trust type system, quantitative trust via continuous semirings, industrial-scale evaluation.

References

- [1] P. Martin-Löf. *Intuitionistic Type Theory*. Bibliopolis, 1984.
- [2] T. Coquand and C. Huet. The Calculus of Constructions. *Inf. Comput.*, 76(2–3):95–120, 1988.
- [3] G. Gentzen. Untersuchungen über das logische Schließen. *Math. Zeitschrift*, 39:176–210, 1935.
- [4] L. de Moura and S. Ullrich. The Lean 4 theorem prover and programming language. In *CADE-28*, LNCS 12699, pp. 625–635. Springer, 2021.
- [5] The Coq Development Team. *The Coq Reference Manual*, version 8.18, 2023.
- [6] N. Swamy et al. Dependent types and multi-monadic effects in F*. In *POPL*, pp. 256–270. ACM, 2016.
- [7] K. R. M. Leino. Dafny: An automatic program verifier. In *LPAR-16*, LNCS 6355, pp. 348–370. Springer, 2010.
- [8] C. A. R. Hoare. An axiomatic basis for computer programming. *CACM*, 12(10):576–580, 1969.
- [9] J. C. Reynolds. Separation logic. In *LICS*, pp. 55–74. IEEE, 2002.
- [10] R. Jung et al. Iris from the ground up. *J. Funct. Program.*, 28:e20, 2018.
- [11] T. Nipkow, L. C. Paulson, and M. Wenzel. *Isabelle/HOL*. LNCS 2283. Springer, 2002.
- [12] N. Vazou et al. Refinement types for Haskell. In *ICFP*, pp. 269–282. ACM, 2014.
- [13] G. Klein et al. seL4: Formal verification of an OS kernel. In *SOSP*, pp. 207–220. ACM, 2009.
- [14] R. Gu et al. CertiKOS. In *OSDI*, pp. 653–669. USENIX, 2016.
- [15] P. Buneman, S. Khanna, and W.-C. Tan. Why and where: Data provenance. In *ICDT*, LNCS 1973, pp. 316–330. Springer, 2001.
- [16] T. J. Green, G. Karvounarakis, and V. Tannen. Provenance semirings. In *PODS*, pp. 31–40. ACM, 2007.
- [17] J. Y. Halpern. *Reasoning about Uncertainty*. MIT Press, 2003.
- [18] G. Shafer. *A Mathematical Theory of Evidence*. Princeton Univ. Press, 1976.
- [19] H. Young. The judgment 8-tuple. *Judgment Geometry Papers*, Paper 02, 2025.
- [20] OpenAI. GPT-4o system card. 2024.
- [21] P. Cousot and R. Cousot. Abstract interpretation: A unified lattice model for static analysis of programs. In *POPL*, pp. 238–252. ACM, 1977.
- [22] J. Bader, J. Aldrich, and É. Tanter. Gradual program verification. In *VMCAI*, LNCS 10747, pp. 25–46. Springer, 2018.
- [23] J. Wise, J. Bader, C. Wong, J. Aldrich, É. Tanter, and J. Sunshine. Gradual verification of recursive heap data structures. In *OOPSLA*, Article 228. ACM, 2020.

- [24] A. Jøsang and R. Ismail. The beta reputation system. In *Bled eConference*, pp. 2502–2511, 2002.
- [25] L. Mui, M. Mohtashemi, and A. Halberstadt. A computational model of trust and reputation. In *HICSS*, pp. 2431–2439. IEEE, 2002.
- [26] A. Jøsang. *Subjective Logic*. Springer, 2016.
- [27] S. D. Kamvar, M. T. Schlosser, and H. Garcia-Molina. The EigenTrust algorithm for reputation management. In *WWW*, pp. 640–651. ACM, 2003.
- [28] L. Xiong and L. Liu. PeerTrust: Supporting reputation-based trust for peer-to-peer electronic communities. *IEEE Trans. Knowledge and Data Eng.*, 16(7):843–857, 2004.
- [29] X. Leroy. Formal verification of a realistic compiler. *CACM*, 52(7):107–115, 2009.

A Per-Program Benchmark Breakdown

Table 11 and Table 12 present representative programs from the 130-program benchmark with their trust levels, H^1 values, and inconsistency classification.

Table 11: Representative clean programs (10 of 100).

#	Program	Trust	H^1	Coords	Props
1	merge_sort	COPILLOT	0	3	7
2	quicksort	COPILLOT	0	3	6
3	heap_sort	COPILLOT	0	3	6
4	binary_search	COPILLOT	0	2	5
5	linked_list	COPILLOT	0	4	8
6	bst_insert	COPILLOT	0	3	6
7	hash_table	ORACLE	0	4	7
8	bfs_traversal	COPILLOT	0	3	5
9	factorial	COPILLOT	0	2	4
10	prime_sieve	COPILLOT	0	2	5

Table 12: Representative inconsistent programs (10 of 30).

#	Program	Trust	H^1	Coords	Bug type
101	merge_sort_bad	CONTRADICTED	1	3	wrong cmp
102	quicksort_bad	COPILLOT	1	3	bad pivot
103	linked_list_loop	COPILLOT	2	4	infinite loop
104	bst_search_bad	COPILLOT	1	3	wrong branch
105	fibonacci_off1	COPILLOT	1	2	off-by-one
106	prime_incomplete	COPILLOT	1	2	missing check
107	palindrome_case	COPILLOT	0	2	case-sensitive
108	gcd_no_base	CONTRADICTED	1	2	no base case
109	stack_no_underflow	COPILLOT	1	3	no underflow
110	radix_bad_digit	COPILLOT	1	3	wrong digit

Observations. Programs 101 and 108 receive **CONTRADICTED** because the SMT solver found counterexamples to their core specifications. Program 107 (palindrome with case-sensitivity bug) has $H^1 = 0$ because the bug does not create a cohomological obstruction—the covering sections glue consistently, but the glued section is simply wrong. This illustrates the limitation of H^1 as an inconsistency detector: it detects structural inconsistencies between sections, not semantic errors within a single section.

Programs 7 (hash_table) and 102 (quicksort_bad) both have trust level **COPILLOT**, but differ in their H^1 values. The clean hash table has $H^1 = 0$, indicating consistent gluing, while the buggy quicksort has $H^1 = 1$, indicating a structural contradiction between the partition specification and the recursive sort specification.

Table 13: Aggregate statistics by program category.

Category	Mean trust	Mean H^1	Mean coords	Mean props
Sorting (clean)	2.0	0.00	3.0	6.3
Data struct (clean)	2.2	0.03	3.5	6.8
Graph (clean)	2.0	0.00	3.1	5.4
Numeric (clean)	2.0	0.05	2.2	4.6
String (clean)	2.0	0.00	2.4	4.9
All inconsistent	1.0	0.87	2.8	5.6